GPU Scheduling and Levelization for Circuit Task Graph Execution

Jianing Ou
jou32@wisc.edu
University of Wisconsin-Madison

Xuechun Jin
xjin252@wisc.edu
University of Wisconsin-Madison

Yayen Lin
lin383@wisc.edu
University of Wisconsin-Madison

Matthew Kearney
mtkearney@wisc.edu
University of Wisconsin-Madison

Abstract

Circuit workloads used in GPU logic simulation contain many gate operations with very small computation and strict dependencies. Based on ISCAS-style circuit benchmarks, we build circuit task graphs and evaluate two execution approaches. The first approach focuses on gate-level scheduling. Under both batch blocking and batch non-blocking execution modes, we implement four scheduling policies: FIFO, fan-in priority, dependency-aware scheduling, and shortest-job-first. The second approach uses levelization and fused levelization, where the execution unit is changed from gate batches to topological levels or fused levels. Our results show that gate-level schedulers are not a good fit for circuit graphs because of kernel launch overhead and the high cost of host-side dependency updates. In contrast, levelization greatly reduces the number of scheduling units and the cost of dependency updates, leading to significant performance improvements on small, medium, and large circuits. Fused levelization further improves performance by reducing repeated kernel launches caused by narrow tail levels. Overall, for circuit simulation, choosing the right execution granularity is more useful than designing a more complex gate-level scheduler.

1 Introduction

A circuit used for logic simulation can naturally be represented as a directed acyclic graph (DAG) [1, 2]. In this graph, each node represents a logic gate operation, such as an AND gate, and each edge represents a signal dependency. A gate can only be executed after all of its predecessor gates have finished. Gates at the same dependency level can theoretically be evaluated in parallel, so GPUs can provide large-scale parallelism and improve performance. However, each single gate operation requires very little computation. Therefore, launching too many small GPU kernels can introduce high kernel launch overhead and synchronization overhead, which may hurt performance.

This project compares two ways to execute circuit graphs (Figure 1). The first method is gate-level scheduling. In this method, each gate is treated as a schedulable task, and the scheduler selects ready gates from the ready queue. The second method is levelized execution where the circuit is first organized into topological levels, and all gates in the same level are executed together as a more coarse-grained task. Since many circuits have later levels with only a few gates, we further implement fused levelization, which merges consecutive small levels into a single GPU kernel to reduce repeated launch overhead in narrow tail levels.

The main contributions of this project are as follows. (1) we implement and compare several gate-level scheduling policies, including FIFO, fan-in priority, SJF, and dependency-aware scheduling. We evaluate the effects of batch size and execution mode (including batch blocking and batch non-blocking) on multiple benchmark circuits. (2) we implement levelization and fused levelization with three fusion thresholds, and evaluate their performance on the same benchmarks.

2 Methodology

2.1 Task Graph Construction

We parse each benchmark to extract gate counts, connections, gate types, and fan-in information, and use them to construct a circuit DAG [1]. For gate-level scheduling, each gate becomes one Task whose incoming edges define its dependencies. When the remaining dependency count becomes 0, the task becomes ready. For levelization, the same circuit DAG is grouped into dependency levels: input gates are placed at level 0, and every other gate is assigned to one level higher than the maximum level of its predecessors.

2.2 Scheduling Policies

All four scheduling policies share the same ready queue mechanism. At the beginning, all gates with no predecessor nodes are inserted into the ready queue. Gates are grouped according to the batch size and then sent to the GPU for execution. After each batch finishes, the scheduler checks the successor gates of the completed gates and inserts gates that remaining dependencies becomes zero into the ready queue. The difference among these four policies is only how the same batch of gates inserted into the ready queue ordered in it before being sent to the GPU.

Fan-in represents the number of predecessor of a gate, while fan-out represents the number of successors of a gate.

FIFO is the simplest policy. Gates are executed in the same order they enter the ready queue. No priority is assigned to each gate.

FaninPriority prioritizes gates with larger fan-in, since they represent heavier GPU computation in our benchmark model.

SJF (Smallest Job First) is the opposite of FaninPriority policy. Gates with smaller fan-in are given higher priority, which means smaller tasks are executed first. This can reduce the average task waiting time.

DependencyAware prioritizes gates with larger fan-out, so gates that can unlock more successors are executed earlier to help keep the GPU busy.

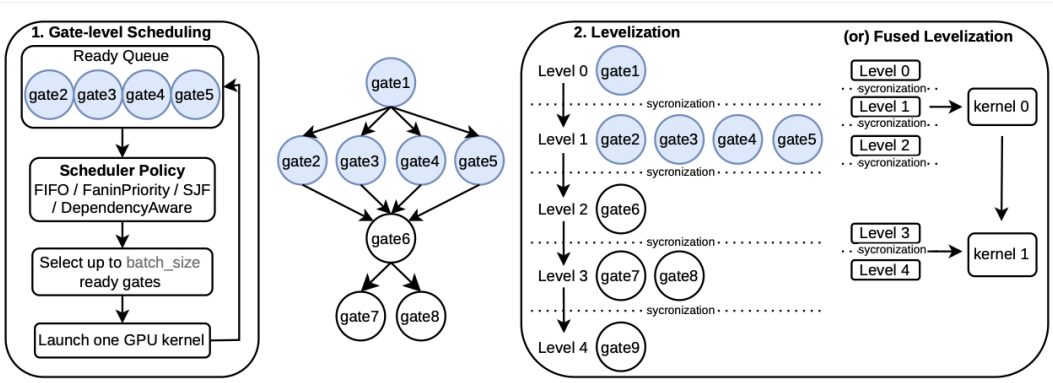


Figure 1: System overview. Suppose the gate1 here in the circuit has been executed, and gate2 / gate3 / gate4 / gate5 is ready to execute.

2.3 Execution Modes

For the above four scheduling policies, we design two execution modes: Batch Blocking and Batch Non-Blocking.

2.3.1 Batch Blocking. Under this mode, the CPU groups a batch of gates and sends them to the GPU for execution, then waits until it finishes before updating dependencies. This creates a batch level barrier: while a batch is running, the CPU only waits and does not send new gates. As a result, the GPU may stay idle during dependency updates, which reduces overall efficiency.

2.3.2 Batch Non-Blocking. Batch Non-Blocking [3] is designed to improve the issue addressed in the Batch Blocking mode. It allows multiple batches to run in parallel on different CUDA streams. The CPU actively checks the status of each stream. As soon as a stream finishes execution, the scheduler immediately launches a new batch and updates dependencies. This helps ensuring there is always a stream running on the GPU at all times, which reduces GPU idle gaps and improve its overall utilization.

2.4 Levelization

2.4.1 Level-by-level. This levelization method [1] groups gates by dependency level. Gates in the same level are independentand can run together, but the next level must wait until the previous level finishes. Therefore, the CPU sends one level at a time to the GPU, with synchronization between levels.

2.4.2 Fused Levelization. Fused levelization [2] merges consecutive small levels on top of the level-by-level method. Given a threshold T, small levels are fused with following levels until a level that contains more than T gates is reached. The fused group is launched as one GPU kernel, while the original level order is preserved using intra-kernel synchronization.

2.5 GPU Executor Backend

Both scheduling and levelization methods use the same GPU executor. After selecting a batch of gates, the host copies their gate IDs to the GPU and launches a CUDA kernel. Each CUDA thread handles one gate, reads its gate type, fan-in, and predecessor outputs, and then computes the gate output. To make gate cost more visible, we repeat the computation based on fan-in, so gates with higher fan-in take longer in our benchmark model [1]. The host records timestamps before and after each kernel launch to calculate host-side makespan and GPU active intervals.

3 Experiment Setup

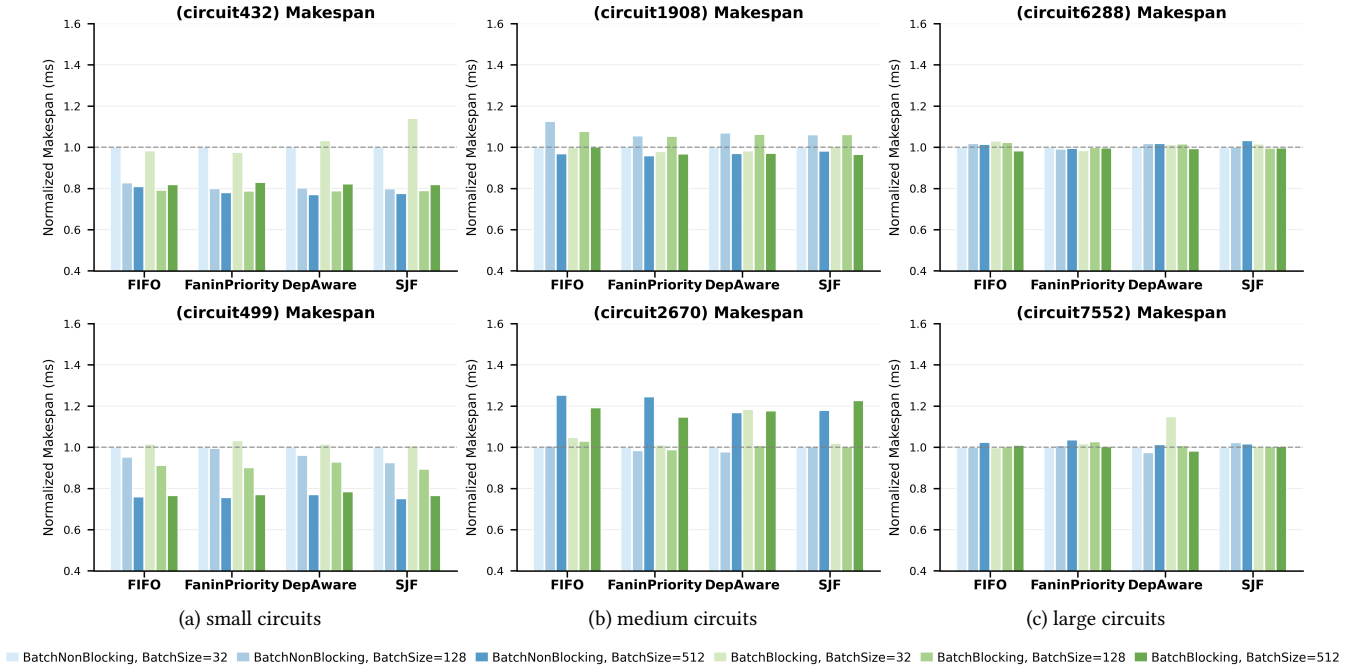
Hardware Configuration. Our experiments are conducted on a Google Colab T4 GPU.
Benchmark. We use 11 circuits [1] in total, which has three sizes: Small (c17, c432, c499, c880), Medium (c1355, c1908, c2670, c3540), and Large (c5315, c6288, c7552).
Measurement. For gate-level scheduling, we evaluate all combinations of four scheduling policies, two execution modes, and three batch sizes (32, 128, and 512). For the Fused Levelization, we choose three fusion threshold T: 256, 1024, and 2048. Each configuration is repeated 10 times and we record the average to reduce variance. For each run, we record the makespan, throughput and GPU utilization.
Metrics. Makespan is defined as the total host-side running time of one scheduler run, from the start until the last GPU task finishes. Throughput is defined as the number of completed gates divided by the makespan. GPU utilization is defined as the union of host-side active GPU intervals divided by the makespan, which is a runtime-level metric rather than profiler-level SM occupancy.

4 Results and Analysis

4.1 Gate-Level Batch Scheduling

We first compare different scheduling policies under the batch blocking mode. We selected several representative circuits here for illustration. As shown by the green bars in Figure 2, we find that circuits such as 499, 6288, and 7552 show very little difference under the same batch size across different scheduling policies. Some other circuits show slightly larger difference, but its overall small.

We believe this is because each gate operation is very lightweight. Simply changing the ordering of the ready queue is usually not enough to overcome the overhead from kernel launches and dependency updates.



(a) small circuits (b) medium circuits (c) large circuits

Figure 2: Gate-level scheduling across small, medium, and large circuits. Each bar is normalized to the FIFO batch blocking baseline with batch size 32.

The DependencyAware policy performs poorly on some circuits. We suspect it is because it quickly unlocks a large number of successor gates and makes the ready queue very large. Then the scheduler needs to performs an O(n) scan over the queue each time, making the scheduling overhead higher.

Another observation we have is that a larger batch size does not always improve performance. For small circuits (Figure 2 (a)), increasing the batch size can reduce the number of kernel launches. Since each gate computation is very lightweight, launch overhead is still a major bottleneck in small circuits, so a larger batch can help reduce this cost sometimes.

However, we also find that in some circuits (Figure 2 (b), (c)), when the batch size increases from 128 to 512, the makespan of different scheduling policies actually becomes larger. We believe this may happen because the system needs to update a larger number of dependencies after each batch finishes, and this overhead can offset or even exceed the reduction in launch overhead. In addition, when the ready gates number is smaller than the batch size, the scheduler has to wait until enough gates are available before launching them, which may waste GPU resources.

The non-blocking execution mode allows schedulers to move on without waiting the current batch to finish. By using multiple CUDA streams in parallel, it can theoretically reduce CPU idle time and lower the overall makespan.

However, surprisingly, the experimental results show that the difference between blocking and non-blocking is still small, as shown by the comparison between the blue and green bars in Figure 2. We believe one reason is that the non-blocking mode improves efficiency mainly by overlapping the execution time of multiple tasks. However, since each gate runs for a very short time, the amount of overlap is limited. In addition, the overhead on the CPU side, including continuously checking stream status, updating dependencies, and maintaining the ready queue, still dominates the overhead. As a result, introducing multiple streams does not significantly reduce the make span and may even increase the management overhead.

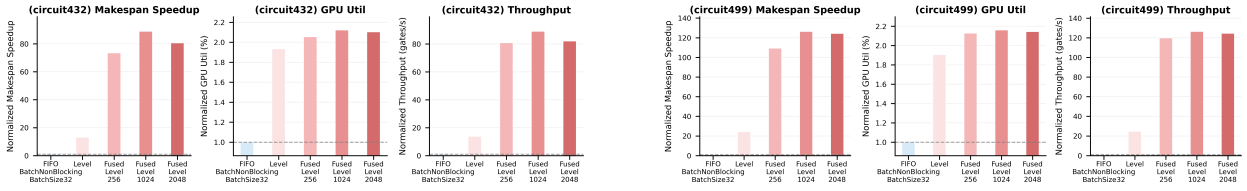
From these results, we can see that treating every gate as an individual task makes the scheduler needs to dynamically maintain the ready queue and frequently update dependencies, which may be the main reason why the optimizations are limited. To truly reduce the overhead, the execution order should be determined in advance and fundamentally avoid dynamic scheduling.

4.2 levelization

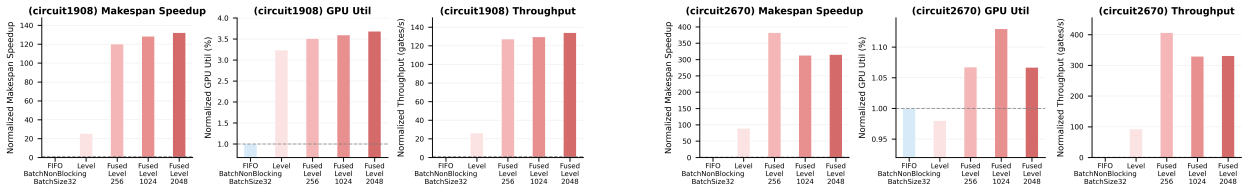
Levelization effectively solves the major problem of gate-level batch scheduling. It no longer needs to maintain a ready queue. Since gates inside the same level are independent, the system can execute the circuit directly according to the topological levels.

From out experimental results (Figure 3), we can see that levelization performs significantly better than previous gate-level scheduling methods for all circuit sizes. Using FIFO with batch non-blocking mode and batch size of 32 as the baseline, levelization achieves about 15x makespan speedup on small circuits and about 500x speedup on large circuits. The speedup becomes larger as the circuit size increases.

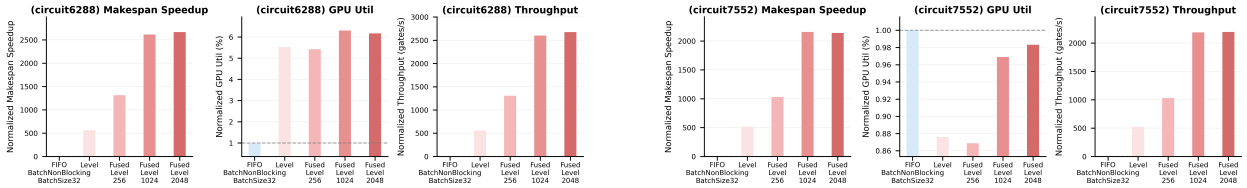
Fused levelization further reduces the number of kernel launches on top of level-by-level levelization. By merging consecutive small levels into a single kernel launch, it further reduces the launch



(a) small circuits



(b) medium circuits



(c) large circuits

Figure 3: Levelization and fused-level execution across small, medium, and large circuits. Each bar is normalized to the FIFO batch non-blocking baseline with batch size 32.

overhead. The results (Figure 3) show that when the threshold is set to 256, the system performances already have clear improvements. However, when the threshold increases to 1024 and 2048, the benefit becomes smaller. We believe this is because most small levels have already been marged when the threshold is 256, so further increasing the threshold does not help much.

5 Weakness and Future Work

Currently, our project still has several limitations. We measure GPU utilization by recording the execution intervals of kernels on the CPU side and dividing the total active time by the overall makespan, instead of using professional profilling tools such as nvprof or Nsight. Therefore, the utilization results may have inaccuracies.

From our results, the performance improvement from levelization is very significant, but it is not magic. When a circuit has a very large number of levels an each level contains only a small number of gates, levelization still needs to launch a kernel for each level. The number of kernel launches is still high, so there is no clear advantage in this case. Similarly, when the circuit is very small, levelization also does not bring much improvement.

Besides this, the thresholds for Fused levelization are selected empirically and are not guaranteed to be optimal for every circuit. Different circuits have very different level distributions, so a fixed threshold may not work well for all cases. In the future, it may be a good idea to dynamically choose the best threshold based on the number of gates each level has of each circuit, which could better improve the performance of fused levelization.

6 Conclusion

This project compares different scheduling policies and execution granularities for GPU logic simulation. Gate-level scheduling is limited by high kernel launch overhead and the high cost of dependency updates. Levelization reduces the scheduling granularity by executing gates level by level. This removes much of the per-gate host-side scheduling and dependency-update overhead, which significantly improves performance. However, narrow tail levels can still cause inefficient small kernel launches. Fused levelization reduces this overhead by executing consecutive small levels within one kernel. Overall, for fine-grained and dependency-heavy GPU applications such as logic simulation, choosing the right execution granularity is usually more important than designing a more complex ready-queue scheduler.

References

[1] Yi-Hua Chung. 2025. LSIM: Logic Simulation Framework. https://github.com/Yi-Huaaa/LSIM. Branch: ECE757.

[2] Yi-Hua Chung, Shui Jiang, Wan-Luan Lee, Yanqing Zhang, Haoxing Ren, Tsung-Yi Ho, and Tsung-Wei Huang. 2025. SimPart: A Simple Yet Effective Replication-Aided Partitioning Algorithm for Logic Simulation on GPU. In *Euro-Par 2025: Parallel Processing: 31st European Conference on Parallel and Distributed Processing, Dresden, Germany, August 25–29, 2025, Proceedings, Part III.*

[3] Dian-Lun Lin, Umit Ogras, Joshua San Miguel, and Tsung-Wei Huang. 2024. TaroRTL: Accelerating RTL Simulation Using Coroutine-Based Heterogeneous Task Graph Scheduling. In *Euro-Par 2024: Parallel Processing: 30th European Conference on Parallel and Distributed Processing, Madrid, Spain, August 26–30, 2024, Proceedings, Part III.*